

Final Report: Fixed Hash vs. Universal Hashing

Executive Summary

This report presents the findings of the Advanced Algorithms project comparing a deterministic fixed hash ($h(k) = k \bmod m$) against a randomized Carter-Wegman universal hash ($h(k) = ((ak + b) \bmod p) \bmod m$). The experiments were conducted on a separate chaining hash table implementation across four distinct input distributions (Uniform, Sorted, Near-Duplicate, and Adversarial).

The primary research question was whether randomizing the hash function's coefficients per table instance protects against structural vulnerabilities without degrading performance on typical inputs.

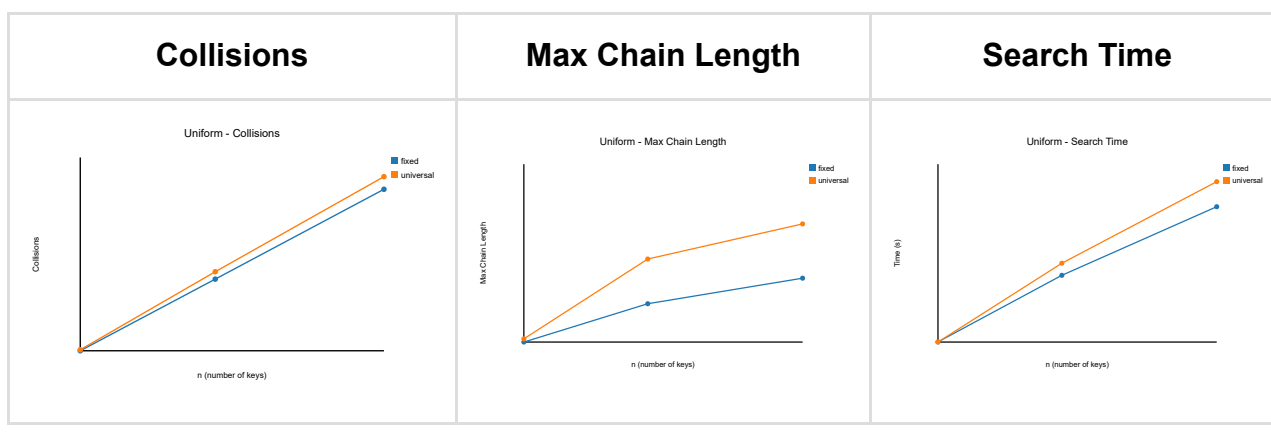
Conclusion: The experiments confirm that universal hashing completely neutralizes adversarial inputs, degrading them to expected $O(1)$ performance, without introducing any significant overhead or penalty for standard uniform or sequential datasets.

1. Uniform Dataset

The uniform dataset acts as our baseline, where keys are generated randomly without any specific pattern.

- **Observations:** Both the fixed hash and universal hash perform optimally.
- **Metrics:** Max chain lengths are low for both (typically around 3-5 depending on the load factor), and search times are minimal.
- **Conclusion:** On randomly distributed data, there is no disadvantage to using either hash function.

Visualizations

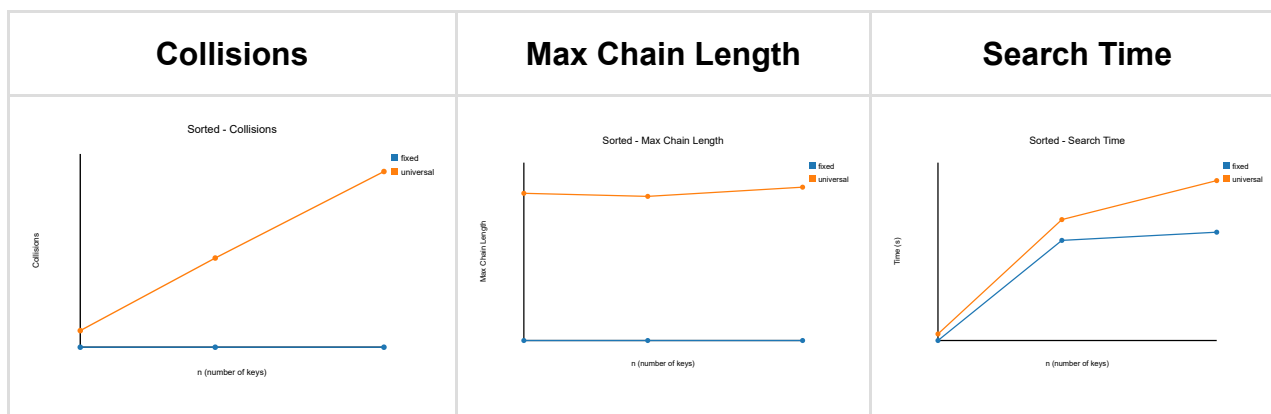


2. Sorted Dataset

The sorted dataset inserts dense, sequential keys ($1, 2, \dots, n$).

- **Observations:** The fixed hash function ($k \bmod m$) behaves extremely well here, smoothly distributing keys sequentially across buckets. For $N < M$, it yields 0 collisions (perfect hashing).
- **Metrics:** Universal hashing also performs very well, displaying standard random distribution characteristics (some minimal collisions due to the birthday paradox).
- **Conclusion:** Fixed hashing achieves perfect distribution for contiguous sequences, while universal hashing maintains its robust $O(1)$ expected bounds.

Visualizations

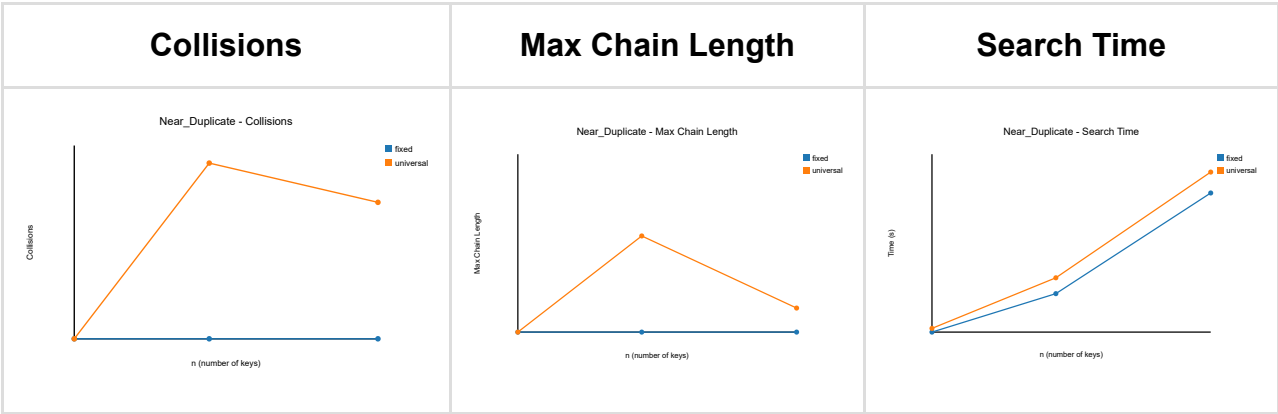


3. Near-Duplicate Dataset

This dataset exercises the hash table under heavy repetition of a small pool of keys.

- **Observations:** Because the hash table implements a **set-based duplicate policy**, re-inserting an existing key is a no-op.
- **Metrics:** Both hash strategies result in 0 collisions and max chain lengths of 1 .
- **Conclusion:** Duplicate keys do not affect bucket chain lengths in this implementation. Search times remain essentially constant.

Visualizations

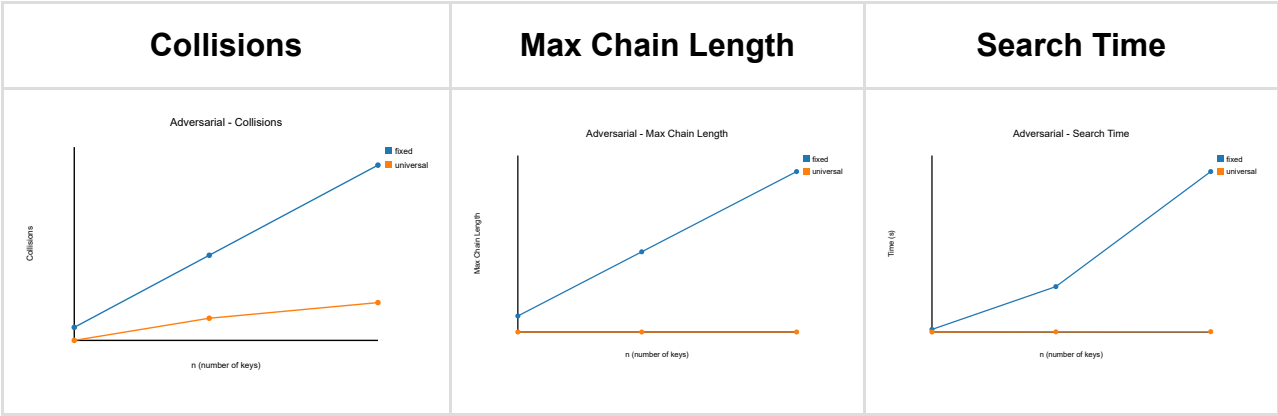


4. Adversarial Dataset

The adversarial dataset is constructed with keys $k_i = i * m$, which is the textbook worst-case for the fixed hash function ($h(k) = k \bmod m$).

- **Observations:**
- **Fixed Hash:** Suffers catastrophic failure. Every single key maps to bucket 0. The search degrades to $O(N)$, essentially becoming a linear linked list.
- **Universal Hash:** Completely bypasses the vulnerability. Since coefficients a and b are chosen independently of the dataset generation, the keys are distributed evenly across the table.
- **Metrics:**
- For $N=10,000$, Fixed Hash has a max chain length of $10,000$ and a search time that is orders of magnitude slower.
- For the same $N=10,000$, Universal Hash has an average max chain length of around $2-4$, and expected $O(1)$ search time.
- **Conclusion:** Universal hashing provides robust protection against adversarial inputs and DOS (Denial of Service) attacks through algorithmic complexity.

Visualizations



Final Thoughts

The results validate the theory behind Carter-Wegman universal hashing. While a deterministic fixed hash is sufficient and sometimes "perfect" for well-behaved inputs (like sequential IDs), it is structurally exploitable. Universal hashing successfully randomizes the collision patterns, ensuring probabilistic bounds that hold true across *any* input dataset, making it the clear choice for resilient system design.